

```

import { useState, useEffect, useRef, useCallback } from 'react'
import { supabase, MOD_PASSWORD } from './supabase'

const G = {
  bg: '#050508', bg2: '#08080f', panel: '#0f0f1a', panel2: '#141428',
  border: '#1a1a30', border2: '#222240',
  accent: '#2A6BFF', success: '#00e5a0', danger: '#ff3366', warning: '#ffb800',
  text: '#eaeaf5', text2: '#7878a0', text3: '#3a3a60',
}

const css = `
  *, *::before, *::after { box-sizing: border-box; margin: 0; padding: 0; }
  html, body, #root { height: 100%; }
  body { background: ${G.bg}; color: ${G.text}; font-family: 'Syne', sans-serif; }
  body::before { content: ''; position: fixed; inset: 0; pointer-events: none; z-
::-webkit-scrollbar { width: 3px; } ::-webkit-scrollbar-thumb { background: ${G
@keyframes fadeIn { from { opacity: 0; transform: translateY(10px); } to { opac
@keyframes slideUp { from { opacity: 0; transform: translateY(30px); } to { opa
@keyframes pulse { 0%,100% { opacity: 1; } 50% { opacity: 0.5; } }
@keyframes glow { 0%,100% { box-shadow: 0 0 10px rgba(42,107,255,0.3); } 50% {
@keyframes called { 0% { transform: scale(0.8); opacity: 0; } 60% { transform:
@keyframes timerPulse { 0%,100% { color: ${G.danger}; } 50% { color: #ff6688; }
@keyframes summaryIn { 0% { opacity: 0; transform: scale(0.95); } 100% { opacit
.fade-in { animation: fadeIn 0.35s ease forwards; }

/* ENTRY */
.entry-wrap { min-height: 100vh; display: flex; align-items: center; justify-co
.entry-box { width: 100%; max-width: 420px; background: ${G.panel}; border: 1px
.entry-box::before { content: ''; position: absolute; top: 0; left: 0; right: 0
.entry-logo { font-family: 'JetBrains Mono', monospace; font-size: 40px; font-w
.entry-logo span { color: ${G.accent}; text-shadow: 0 0 20px rgba(42,107,255,0.
.entry-sub { font-family: 'JetBrains Mono', monospace; font-size: 10px; letter-
.entry-label { font-family: 'JetBrains Mono', monospace; font-size: 10px; lette
.entry-input { width: 100%; background: ${G.bg2}; border: 1px solid ${G.border}
.entry-input:focus { border-color: ${G.accent}; box-shadow: 0 0 0 3px rgba(42,1
.entry-btn { width: 100%; border: none; padding: 15px; font-size: 13px; font-we
.entry-btn-primary { background: ${G.accent}; color: #fff; } .entry-btn-primary
.entry-err { color: ${G.danger}; font-size: 11px; margin-top: 10px; font-family
.entry-tabs { display: flex; gap: 0; margin-bottom: 28px; border-bottom: 1px so
.entry-tab { flex: 1; padding: 10px; text-align: center; font-family: 'JetBrain
.entry-tab.active { color: ${G.accent}; border-bottom-color: ${G.accent}; }

/* PARTICIPANT */
.pview { min-height: 100vh; background: ${G.bg}; }

```

```

.pview-header { background: ${G.panel}; border-bottom: 1px solid ${G.border}; p
.pview-logo { font-family: 'JetBrains Mono', monospace; font-size: 20px; font-w
.pview-logo span { color: ${G.accent}; }
.pview-content { padding: 20px; }

/* CALLED */
.called-screen { position: fixed; inset: 0; background: ${G.bg}; display: flex;
.called-icon { font-size: 80px; margin-bottom: 24px; animation: pulse 1s infini
.called-title { font-size: 32px; font-weight: 800; color: ${G.success}; margin-
.called-name { font-size: 20px; color: ${G.text2}; text-align: center; font-fam
.called-hint { font-size: 14px; color: ${G.text2}; margin-top: 24px; text-align

/* VOTE */
.vote-card { background: ${G.panel}; border: 1px solid ${G.border}; padding: 24
.vote-card.active-vote { border-color: ${G.accent}; box-shadow: 0 0 20px rgba(4
.vote-question { font-size: 18px; font-weight: 700; margin-bottom: 20px; line-h
.vote-buttons { display: grid; grid-template-columns: 1fr 1fr 1fr; gap: 10px; }
.vote-btn { padding: 16px 8px; border: 2px solid; font-size: 13px; font-weight:
.vote-btn-ano { border-color: ${G.success}; color: ${G.success}; background: tr
.vote-btn-ano:hover, .vote-btn-ano.selected { background: ${G.success}; color:
.vote-btn-ne { border-color: ${G.danger}; color: ${G.danger}; background: trans
.vote-btn-ne:hover, .vote-btn-ne.selected { background: ${G.danger}; color: #ff
.vote-btn-zdrzuji { border-color: ${G.warning}; color: ${G.warning}; background
.vote-btn-zdrzuji:hover, .vote-btn-zdrzuji.selected { background: ${G.warning};
.vote-voted { text-align: center; padding: 16px; font-family: 'JetBrains Mono',

/* SPEAKER */
.speaker-card { background: ${G.panel}; border: 1px solid ${G.border}; padding:
.speaker-queue-item { display: flex; align-items: center; gap: 12px; padding: 1

/* REACTIONS */
.reaction-bar { display: flex; gap: 10px; flex-wrap: wrap; }
.reaction-btn { padding: 10px 16px; border: 1px solid ${G.border}; background:
.reaction-btn:hover { border-color: ${G.accent}; transform: scale(1.1); }
.reaction-btn.active-reaction { border-color: ${G.accent}; background: rgba(42,

/* AGENDA */
.agenda-item { padding: 14px 18px; display: flex; align-items: center; gap: 14p
.agenda-item.active-agenda { border-color: ${G.accent}; background: rgba(42,107
.agenda-num { font-family: 'JetBrains Mono', monospace; font-size: 18px; font-w
.agenda-item.active-agenda .agenda-num { color: ${G.accent}; }

/* DISPLAY */
.display { min-height: 100vh; background: ${G.bg}; display: flex; flex-directio
.display-header { padding: 20px 32px; background: ${G.panel}; border-bottom: 1p
.display-logo { font-family: 'JetBrains Mono', monospace; font-size: 28px; font
.display-logo span { color: ${G.accent}; }

```

```

.display-content { flex: 1; padding: 32px; display: grid; grid-template-columns
.display-main { display: flex; flex-direction: column; gap: 20px; }
.display-side { display: flex; flex-direction: column; gap: 16px; }
.result-card { background: ${G.panel}; border: 1px solid ${G.border}; padding:
.result-question { font-size: 24px; font-weight: 800; margin-bottom: 28px; line
.result-bar-wrap { margin-bottom: 20px; }
.result-bar-label { display: flex; justify-content: space-between; align-items:
.result-bar-track { height: 16px; background: ${G.border}; position: relative;
.result-bar-fill { height: 100%; transition: width 0.8s cubic-bezier(0.4,0,0.2,
.result-voters { display: flex; flex-wrap: wrap; gap: 8px; margin-top: 12px; }
.result-voter { font-family: 'JetBrains Mono', monospace; font-size: 11px; padd
.voter-ano { background: rgba(0,229,160,0.12); color: ${G.success}; }
.voter-ne { background: rgba(255,51,102,0.12); color: ${G.danger}; }
.voter-zdrzuji { background: rgba(255,184,0,0.1); color: ${G.warning}; }
.display-speaker-card { background: ${G.panel}; border: 1px solid ${G.border};
.display-called-card { background: rgba(0,229,160,0.05); border: 2px solid ${G.
.display-called-name { font-size: 36px; font-weight: 800; color: ${G.success};
.display-called-label { font-family: 'JetBrains Mono', monospace; font-size: 11
.timer-display { font-family: 'JetBrains Mono', monospace; font-size: 64px; fon
.timer-green { color: ${G.success}; }
.timer-yellow { color: ${G.warning}; }
.timer-red { color: ${G.danger}; animation: timerPulse 0.5s infinite; }
.reactions-display { display: flex; flex-wrap: wrap; gap: 10px; }
.reaction-pill { display: flex; align-items: center; gap: 6px; padding: 8px 14p
.participants-badge { display: flex; align-items: center; gap: 8px; }
.participants-dot { width: 8px; height: 8px; border-radius: 50%; background: ${

/* SUMMARY SCREEN */
.summary-screen { position: fixed; inset: 0; background: rgba(5,5,8,0.97); z-in
.summary-title { font-family: 'JetBrains Mono', monospace; font-size: 14px; let
.summary-name { font-size: 40px; font-weight: 800; margin-bottom: 8px; }
.summary-grid { display: grid; grid-template-columns: repeat(3, 1fr); gap: 20px
.summary-stat { background: ${G.panel}; border: 1px solid ${G.border}; padding:
.summary-stat-val { font-family: 'JetBrains Mono', monospace; font-size: 48px;
.summary-stat-label { font-family: 'JetBrains Mono', monospace; font-size: 10px
.summary-resolutions { width: 100%; max-width: 900px; }
.summary-resolution { padding: 16px 20px; background: ${G.panel}; border: 1px s
.summary-resolution-num { font-family: 'JetBrains Mono', monospace; font-size:
.summary-resolution-text { font-size: 14px; flex: 1; }
.summary-resolution-result { font-family: 'JetBrains Mono', monospace; font-siz
.result-prijat { color: ${G.success}; }
.result-zamitnout { color: ${G.danger}; }
.result-nerozhodnut { color: ${G.warning}; }
.summary-countdown { font-family: 'JetBrains Mono', monospace; font-size: 12px;

/* MODERATOR */
.mod-wrap { min-height: 100vh; background: ${G.bg}; }

```

```
.mod-header { background: ${G.panel}; border-bottom: 1px solid ${G.border}; padding: 10px 0; }
.mod-logo { font-family: 'JetBrains Mono', monospace; font-size: 22px; font-weight: bold; }
.mod-logo span { color: ${G.accent}; }
.mod-badge { background: ${G.danger}; color: #fff; font-family: 'JetBrains Mono', monospace; font-size: 12px; padding: 2px 5px; }
.mod-content { padding: 24px; display: grid; grid-template-columns: 1fr 1fr; gap: 16px; }
.mod-panel { background: ${G.panel}; border: 1px solid ${G.border}; padding: 20px; }
.mod-title { font-family: 'JetBrains Mono', monospace; font-size: 9px; letter-spacing: 0.5px; }
.mod-title::after { content: ''; flex: 1; height: 1px; background: ${G.border}; }
.reaction-mod-item { display: flex; align-items: center; gap: 10px; padding: 8px 16px; }

```

```
/* FORMS */

```

```
.fi { width: 100%; background: ${G.bg2}; border: 1px solid ${G.border}; color: ${G.text}; padding: 10px 16px; }
.fi:focus { border-color: ${G.accent}; }
.fs { width: 100%; background: ${G.bg2}; border: 1px solid ${G.border}; color: ${G.text}; padding: 10px 16px; }
.btn { border: none; padding: 10px 20px; font-size: 11px; font-weight: 700; letter-spacing: 0.5px; }
.btn-p { background: ${G.accent}; color: #fff; } .btn-p:hover { background: #1a202c; }
.btn-s { background: ${G.success}; color: #000; } .btn-s:hover { opacity: 0.85; }
.btn-d { background: ${G.danger}; color: #fff; } .btn-d:hover { opacity: 0.85; }
.btn-w { background: ${G.warning}; color: #000; } .btn-w:hover { opacity: 0.85; }
.btn-g { background: transparent; border: 1px solid ${G.border}; color: ${G.text}; }
.btn-g:hover { border-color: ${G.text2}; color: ${G.text}; }
.btn-full { width: 100%; margin-bottom: 8px; }
.call-btn { padding: 10px 16px; background: ${G.success}; color: #000; border: 1px solid #000; }
.call-btn:hover { opacity: 0.85; }
.done-btn { padding: 10px 16px; background: transparent; border: 1px solid ${G.border}; }
.done-btn:hover { border-color: ${G.danger}; color: ${G.danger}; }

```

```
/* HISTORY */

```

```
.history-item { padding: 16px; background: ${G.panel}; border: 1px solid ${G.border}; margin-bottom: 16px; }

```

```
/* UTILS */

```

```
.loading { display: flex; align-items: center; justify-content: center; min-height: 100px; }
.sec { font-family: 'JetBrains Mono', monospace; font-size: 9px; letter-spacing: 0.5px; }
.sec::after { content: ''; flex: 1; height: 1px; background: ${G.border}; }
.div { height: 1px; background: ${G.border}; margin: 16px 0; }
.participants-count { font-family: 'JetBrains Mono', monospace; font-size: 12px; font-weight: bold; }

```

```
@media (max-width: 900px) {
  .mod-content { grid-template-columns: 1fr; }
  .display-content { grid-template-columns: 1fr; }
  .vote-buttons { grid-template-columns: 1fr; }
  .timer-display { font-size: 48px; }
  .summary-grid { grid-template-columns: 1fr 1fr; }
}

```

```
const formatTimer = (s) => `${Math.floor(s/60)}:${String(s%60).padStart(2, '0')}`

```

```

const getResolutionResult = (poll) => {
  const votes = poll.vote_responses || []
  const ano = votes.filter(v => v.vote === 'ano').length
  const ne = votes.filter(v => v.vote === 'ne').length
  if (ano > ne) return { text: 'PŘIJATO', cls: 'result-prijat' }
  if (ne > ano) return { text: 'ZAMÍTNUTO', cls: 'result-zamitnout' }
  return { text: 'NEROZHODNUTO', cls: 'result-nerozhodnut' }
}

// -- PDF EXPORT --
const generatePDF = (session, polls, speakers, agenda, participants) => {
  const now = new Date()
  const dateStr = now.toLocaleDateString('cs-CZ', { day: 'numeric', month: 'long' })
  const timeStr = now.toLocaleTimeString('cs-CZ', { hour: '2-digit', minute: '2-digit' })
  const doneSpeakers = speakers.filter(s => s.status === 'done' || s.status === 'closed')
  const closedPolls = polls.filter(p => p.status === 'closed')

  const html = `<!DOCTYPE html>
<html lang="cs">
<head>
<meta charset="utf-8">
<title>Protokol - ${session.title}</title>
<style>
  * { box-sizing: border-box; margin: 0; padding: 0; }
  body { font-family: 'Arial', sans-serif; color: #111; background: #fff; padding: 20px; }
  .header { border-bottom: 3px solid #111; padding-bottom: 20px; margin-bottom: 20px; }
  .header-logo { font-size: 28px; font-weight: 900; letter-spacing: -1px; margin-bottom: 10px; }
  .header-logo span { color: #2A6BFF; }
  .header-title { font-size: 20px; font-weight: 700; margin-bottom: 4px; }
  .header-meta { font-size: 12px; color: #666; }
  .section { margin-bottom: 28px; }
  .section-title { font-size: 11px; letter-spacing: 3px; text-transform: uppercase; }
  .stats-row { display: flex; gap: 20px; margin-bottom: 20px; }
  .stat-box { border: 1px solid #ddd; padding: 14px 20px; flex: 1; text-align: center; }
  .stat-val { font-size: 32px; font-weight: 900; }
  .stat-label { font-size: 10px; text-transform: uppercase; letter-spacing: 1px; }
  .agenda-item { padding: 8px 12px; border-left: 3px solid #ddd; margin-bottom: 6px; }
  .agenda-item.active { border-left-color: #2A6BFF; }
  .poll-block { border: 1px solid #ddd; padding: 16px; margin-bottom: 14px; }
  .poll-question { font-size: 14px; font-weight: 700; margin-bottom: 10px; }
  .poll-resolution { font-size: 13px; font-weight: 700; margin-bottom: 8px; padding: 0 10px; }
  .res-prijato { background: #e8fdf5; color: #00a36c; }
  .res-zamitnuto { background: #fff0f3; color: #cc2244; }
  .res-nerozhodnut { background: #fffbf0; color: #cc7700; }
  .vote-row { display: flex; gap: 20px; margin-bottom: 8px; }
  .vote-count { font-size: 13px; font-weight: 700; }

```

```

.vote-ano { color: #00a36c; }
.vote-ne { color: #cc2244; }
.vote-zdrzuj { color: #cc7700; }
.voters-list { font-size: 11px; color: #555; margin-top: 6px; }
.speaker-item { padding: 8px 12px; border-bottom: 1px solid #f0f0f0; display: flex; align-items: center; }
.speaker-num { color: #2A6BFF; font-weight: 700; min-width: 24px; }
.participant-list { display: flex; flex-wrap: wrap; gap: 8px; }
.participant-tag { background: #f5f5f5; padding: 4px 10px; font-size: 11px; border: 1px solid #ddd; font-weight: 700; }
.footer { margin-top: 40px; padding-top: 16px; border-top: 1px solid #ddd; font-size: 11px; }

@media print { body { padding: 20px; } }
</style>
</head>
<body>
<div class="header">
  <div class="header-logo">[<span>CTRL</span>]</div>
  <div class="header-title">Protokol z konference</div>
  <div style="font-size:18px; font-weight:800; margin:6px 0;">${session.title}</div>
  <div class="header-meta">Datum: ${dateStr} . Čas vygenerování: ${timeStr} . PIN: ${pin}</div>
</div>

<div class="section">
  <div class="section-title">Souhrn</div>
  <div class="stats-row">
    <div class="stat-box"><div class="stat-val">${participants.length}</div><div class="stat-val">${closedPolls.length}</div><div class="stat-val">${doneSpeakers.length}</div><div class="stat-val">${agenda.length}</div></div>
  </div>

  ${agenda.length > 0 ? `
<div class="section">
  <div class="section-title">Program jednání</div>
  ${agenda.map(a => `<div class="agenda-item${a.active ? ' active' : ''}><strong>${a.title}</strong> ${a.time}</div>` : ''}
</div>` : ''}

  ${closedPolls.length > 0 ? `
<div class="section">
  <div class="section-title">Výsledky hlasování a usnesení</div>
  ${closedPolls.map((poll, i) => {
    const votes = poll.vote_responses || []
    const ano = votes.filter(v => v.vote === 'ano').length
    const ne = votes.filter(v => v.vote === 'ne').length
    const zdrzuj = votes.filter(v => v.vote === 'zdrzuj_se').length
    const res = getResolutionResult(poll)
    const anoVoters = votes.filter(v => v.vote === 'ano').map(v => v.voter_name).join(' ')
    const neVoters = votes.filter(v => v.vote === 'ne').map(v => v.voter_name).join(' ')
    const zdrzujVoters = votes.filter(v => v.vote === 'zdrzuj_se').map(v => v.voter_name).join(' ')
    const resText = res ? res : 'Nebylo dosaženo usnesení'
    `<div class="poll-item">
      <div class="poll-title">${poll.title}</div>
      <div class="poll-info">
        <div class="poll-ano">ano: ${ano}</div>
        <div class="poll-ne">ne: ${ne}</div>
        <div class="poll-zdrzuj">zdrzuj: ${zdrzuj}</div>
        <div class="poll-res">${resText}</div>
      </div>
      <div class="poll-voters">
        <div class="poll-ano-voters">ano: ${anoVoters}</div>
        <div class="poll-ne-voters">ne: ${neVoters}</div>
        <div class="poll-zdrzuj-voters">zdrzuj: ${zdrzujVoters}</div>
      </div>
    </div>` : ''}
  )}
  </div>` : ''}
  </div>
</div>

```

```

const zdrzVoters = votes.filter(v => v.vote === 'zdrzuji_se').map(v => v.vote
return `<div class="poll-block">
  <div style="font-size:11px;color:#999;letter-spacing:1px;text-transform:upp
  <div class="poll-question">${poll.question}</div>
  <div class="poll-resolution res-${res.text === 'PŘIJATO' ? 'prijato' : res.
  <div class="vote-row" style="margin-top:10px">
    <span class="vote-count vote-ano">ANO: ${ano}</span>
    <span class="vote-count vote-ne">NE: ${ne}</span>
    <span class="vote-count vote-zdrzuji">ZDRŽEL SE: ${zdrzuji}</span>
    <span style="color:#999;font-size:12px">Celkem: ${votes.length}</span>
  </div>
  ${anoVoters ? `<div class="voters-list"><strong style="color:#00a36c">ANO:<
  ${neVoters ? `<div class="voters-list"><strong style="color:#cc2244">NE:</s
  ${zdrzVoters ? `<div class="voters-list"><strong style="color:#cc7700">ZDRŽ
  </div>`
  }).join('')}
</div>` : ''}

${doneSpeakers.length > 0 ? `
<div class="section">
  <div class="section-title">Řečníci</div>
  ${doneSpeakers.map((s, i) => `<div class="speaker-item"><div class="speaker-num
</div>` : ''}

<div class="section">
  <div class="section-title">Účastníci (${participants.length})</div>
  <div class="participant-list">
    ${participants.map(p => `<span class="participant-tag">${p.name}</span>`).joi
  </div>
</div>

<div class="footer">
  <span>CTRL Europe Team - Hlasovací systém</span>
  <span>Vygenerováno: ${dateStr} v ${timeStr}</span>
</div>
</body>
</html>`

```

```

const win = window.open('', '_blank')
win.document.write(html)
win.document.close()
setTimeout(() => win.print(), 500)
}

// -- ENTRY --
function EntryScreen({ onParticipant, onModerator, onDisplay }) {
  const [tab, setTab] = useState('participant')

```

```

const [name, setName] = useState('')
const [pin, setPin] = useState('')
const [modPass, setModPass] = useState('')
const [error, setError] = useState('')
const [loading, setLoading] = useState(false)
const [sessions, setSessions] = useState([])
const [selectedSession, setSelectedSession] = useState('')

useEffect(() => {
  supabase.from('vote_sessions').select('*').eq('active', true).then(({ data })
}, [])

const joinAsParticipant = async () => {
  if (!name.trim()) return setError('// Zadej své jméno')
  if (!pin.trim()) return setError('// Zadej PIN konference')
  setLoading(true); setError('')
  const { data: session } = await supabase.from('vote_sessions').select('*').eq
  if (!session) { setError('// Nesprávný PIN nebo konference není aktivní'); se
  const { data: participant } = await supabase.from('vote_participants').insert
  onParticipant({ session, participant })
  setLoading(false)
}

const joinAsModerator = () => {
  if (modPass !== MOD_PASSWORD) return setError('// Nesprávné heslo moderátora'
  onModerator()
}

const openDisplay = () => {
  if (!selectedSession) return setError('// Vyber konferenci')
  const session = sessions.find(s => s.id === parseInt(selectedSession))
  onDisplay(session)
}

return (
  <div className="entry-wrap">
    <div className="entry-box">
      <div className="entry-logo"><span>CTRL</span></div>
      <div className="entry-sub">Hlasovací systém . Conference</div>
      <div className="entry-tabs">
        {[['participant', 'Účastník'], ['display', 'Obrazovka'], ['moderator', 'Mode
          <div key={id} className={`entry-tab${tab===id?' active':''}`} onClick
            )]}
      </div>
      {tab === 'participant' && (
        <div className="fade-in">
          <div className="entry-label">Tvoje jméno</div>

```

```

        <input className="entry-input" placeholder="Jan Novák" value={name} o
        <div className="entry-label">PIN konference</div>
        <input className="entry-input" placeholder="XXXX" value={pin} onChange
        <button className="entry-btn entry-btn-primary" onClick={joinAsPartic
    </div>
  )}
  {tab === 'display' && (
    <div className="fade-in">
      <div className="entry-label">Vyber konferenci</div>
      <select className="fi" value={selectedSession} onChange={e => setSele
        <option value="">-- Vyber konferenci --</option>
        {sessions.map(s => <option key={s.id} value={s.id}>{s.title}</optio
      </select>
      <button className="entry-btn entry-btn-primary" onClick={openDisplay}
    </div>
  )}
  {tab === 'moderator' && (
    <div className="fade-in">
      <div className="entry-label">Heslo moderátora</div>
      <input className="entry-input" type="password" placeholder="*****
      <button className="entry-btn entry-btn-primary" onClick={joinAsModera
    </div>
  )}
  {error && <div className="entry-err">{error}</div>}
</div>
</div>
)
}

```

```
// -- PARTICIPANT --
```

```

function ParticipantView({ session, participant }) {
  const [polls, setPolls] = useState([])
  const [myVotes, setMyVotes] = useState({})
  const [speakers, setSpeakers] = useState([])
  const [agenda, setAgenda] = useState([])
  const [myReaction, setMyReaction] = useState(null)
  const [showSpeakerForm, setShowSpeakerForm] = useState(false)
  const [speakerNote, setSpeakerNote] = useState('')
  const [isCalled, setIsCalled] = useState(false)
  const [requested, setRequested] = useState(false)
  const [sessionClosed, setSessionClosed] = useState(false)

  const load = useCallback(async () => {
    const [{ data: p }, { data: s }, { data: a }, { data: sess }] = await Promise
      supabase.from('vote_polls').select('*', vote_responses('*')).eq('session_id',
      supabase.from('vote_speakers').select('*').eq('session_id', session.id).in(
      supabase.from('vote_agenda').select('*').eq('session_id', session.id).order

```

```

    supabase.from('vote_sessions').select('*').eq('id', session.id).single(),
  ])
  if (p) setPolls(p)
  if (s) setSpeakers(s)
  if (a) setAgenda(a)
  if (sess && !sess.active) setSessionClosed(true)
  const mySpeaker = s?.find(sp => sp.participant_id === participant.id && sp.st
  if (mySpeaker) setIsCalled(true)
  const myReq = s?.find(sp => sp.participant_id === participant.id && sp.status
  if (myReq) setRequested(true)
}, [session.id, participant.id])

useEffect(() => {
  load()
  const channel = supabase.channel(`p-${session.id}-${participant.id}`)
    .on('postgres_changes', { event: '*', schema: 'public', table: 'vote_polls'
    .on('postgres_changes', { event: '*', schema: 'public', table: 'vote_respon
    .on('postgres_changes', { event: '*', schema: 'public', table: 'vote_speake
    .on('postgres_changes', { event: '*', schema: 'public', table: 'vote_sessio
    .subscribe()
  return () => supabase.removeChannel(channel)
}, [load, session.id, participant.id])

const castVote = async (pollId, vote) => {
  if (myVotes[pollId]) return
  await supabase.from('vote_responses').insert([{ poll_id: pollId, voter_name:
  setMyVotes(prev => ({ ...prev, [pollId]: vote }))
}

const requestSpeaker = async () => {
  if (requested) return
  await supabase.from('vote_speakers').insert([{ session_id: session.id, partic
  setRequested(true); setShowSpeakerForm(false); setSpeakerNote('')
}

const sendReaction = async (reaction) => {
  const newReaction = myReaction === reaction ? null : reaction
  setMyReaction(newReaction)
  await supabase.from('vote_participants').update({ reaction: newReaction, reac
}

if (sessionClosed) return (
  <>
  <style>{css}</style>
  <div className="called-screen" style={{ background: G.bg }}>
    <div style={{ fontSize: 60, marginBottom: 20 }}></div>
    <div style={{ fontSize: 28, fontWeight: 800, marginBottom: 8 }}>Konferenc

```

```

        <div style={{ color: G.text2, fontFamily: 'JetBrains Mono, monospace', fo
    </div>
    </>
  )

  if (isCalled) return (
    <>
      <style>{css}</style>
      <div className="called-screen">
        <div className="called-icon"></div>
        <div className="called-title">Jsi vyvolán!</div>
        <div className="called-name">{participant.name}</div>
        <div className="called-hint">Začni mluvit - moderátor spustí časomíru</di
        <button className="btn btn-g" style={{ marginTop: 32 }} onClick={() => se
      </div>
    </>
  )

  const activePolls = polls.filter(p => p.status === 'active')
  const closedPolls = polls.filter(p => p.status === 'closed')
  const activeAgenda = agenda.find(a => a.active)
  const waitingSpeakers = speakers.filter(s => s.status === 'waiting')

  return (
    <>
      <style>{css}</style>
      <div className="pview">
        <div className="pview-header">
          <div><div className="pview-logo">[<span>CTRL</span>]</div><div style={{
          <div style={{ textAlign: 'right' }}>
            <div style={{ fontFamily: 'JetBrains Mono, monospace', fontSize: 12, col
            <div style={{ fontFamily: 'JetBrains Mono, monospace', fontSize: 9, colo
          </div>
        </div>
        <div className="pview-content">
          {activeAgenda && (
            <div style={{ background: 'rgba(42,107,255,0.05)', border: `1px solid r
              <div style={{ fontFamily: 'JetBrains Mono, monospace', fontSize: 9, co
              <div style={{ fontSize: 15, fontWeight: 700 }}>{activeAgenda.title}</
              {activeAgenda.description && <div style={{ fontSize: 12, color: G.tex
            </div>
          )}
          {activePolls.map(poll => (
            <div key={poll.id} className="vote-card active-vote">
              <div style={{ fontFamily: 'JetBrains Mono, monospace', fontSize: 9, co
                <span style={{ animation: 'pulse 1s infinite', display: 'inline-blo
              </div>

```

```

<div className="vote-question">{poll.question}</div>
{myVotes[poll.id] ? (
  <div className="vote-voted">v Hlasoval/a jsi: {myVotes[poll.id].t
) : (
  <div className="vote-buttons">
    {[['ano', 'ANO v', 'vote-btn-ano'], ['ne', 'NE x', 'vote-btn-ne'], ['
      <button key={val} className={`vote-btn ${cls}`} onClick={() =
    )}]
  </div>
)}
</div>
)}}
<div className="speaker-card">
  <div className="sec">REAKCE</div>
  <div className="reaction-bar">
    {[['', 'souhlas'], ['', 'namitka'], ['', 'otazka']].map(([icon, type]) =>
      <button key={type} className={`reaction-btn${myReaction===type?'
    )})
    {myReaction && <button className="btn btn-g" style={{ fontSize:11 }
  </div>
</div>
<div className="speaker-card">
  <div className="sec">PŘIHLÁŠIT SE O SLOVO</div>
  {!requested ? (
    showSpeakerForm ? (
      <div>
        <input className="fi" placeholder="Poznámka k projevu (voliteln
        <div style={{ display:'flex', gap:8 }}>
          <button className="btn btn-p" onClick={requestSpeaker}>PŘIHLÁ
          <button className="btn btn-g" onClick={() => setShowSpeakerFo
        </div>
      </div>
    ) : (
      <button className="btn btn-p btn-full" onClick={() => setShowSpea
    )
  ) : (
    <div style={{ fontFamily:'JetBrains Mono,monospace', fontSize:12, c
  )}}
  {waitingSpeakers.length > 0 && (
    <div style={{ marginTop:16 }}>
      <div style={{ fontFamily:'JetBrains Mono,monospace', fontSize:9,
        {waitingSpeakers.map((s,i) => (
          <div key={s.id} className="speaker-queue-item">
            <div style={{ fontFamily:'JetBrains Mono,monospace', fontSize
              <div><div style={{ fontSize:14, fontWeight:600 }}>{s.name}</d
            </div>
          )})
        )})

```

```

        </div>
      )}
    </div>
    {closedPolls.length > 0 && (
      <div>
        <div className="sec">VÝSLEDKY HLASOVÁNÍ</div>
        {closedPolls.map((poll, i) => {
          const votes = poll.vote_responses || []
          const ano = votes.filter(v => v.vote==='ano').length
          const ne = votes.filter(v => v.vote==='ne').length
          const zdrzují = votes.filter(v => v.vote==='zdrzují_se').length
          const res = getResolutionResult(poll)
          return (
            <div key={poll.id} className="history-item">
              <div style={{ fontFamily:'JetBrains Mono,monospace', fontSize
                <div style={{ fontSize:14, fontWeight:700, marginBottom:8 }}>
                <div style={{ fontFamily:'JetBrains Mono,monospace', fontSize
                <div style={{ display:'flex', gap:16 }}>
                  <span style={{ color:G.success }}>ANO: {ano}</span>
                  <span style={{ color:G.danger }}>NE: {ne}</span>
                  <span style={{ color:G.warning }}>ZDRŽEL: {zdrzují}</span>
                </div>
              </div>
            </div>
          )
        })}
      </div>
    )}
  </div>
</div>
</>
)
}

```

```
// -- DISPLAY --
```

```

function DisplayScreen({ session }) {
  const [polls, setPolls] = useState([])
  const [speakers, setSpeakers] = useState([])
  const [agenda, setAgenda] = useState([])
  const [reactions, setReactions] = useState([])
  const [participants, setParticipants] = useState([])
  const [timerSeconds, setTimerSeconds] = useState(120)
  const [timerRunning, setTimerRunning] = useState(false)
  const [showSummary, setShowSummary] = useState(false)
  const [summaryCountdown, setSummaryCountdown] = useState(60)
  const timerRef = useRef(null)
  const summaryRef = useRef(null)

```

```

const load = useCallback(async () => {
  const [{ data: p }, { data: s }, { data: a }, { data: r }, { data: part }, {
    supabase.from('vote_polls').select('*', vote_responses('*')).eq('session_id',
    supabase.from('vote_speakers').select('*').eq('session_id', session.id).ord
    supabase.from('vote_agenda').select('*').eq('session_id', session.id).order
    supabase.from('vote_participants').select('*').eq('session_id', session.id)
    supabase.from('vote_participants').select('*').eq('session_id', session.id)
    supabase.from('vote_sessions').select('*').eq('id', session.id).single(),
  ]
  if (p) setPolls(p)
  if (s) setSpeakers(s)
  if (a) setAgenda(a)
  if (r) setReactions(r)
  if (part) setParticipants(part)
  if (sess && !sess.active) setShowSummary(true)
}, [session.id])

useEffect(() => {
  load()
  const channel = supabase.channel(`display-${session.id}`)
    .on('postgres_changes', { event: '*', schema: 'public', table: 'vote_polls'
    .on('postgres_changes', { event: '*', schema: 'public', table: 'vote_respon
    .on('postgres_changes', { event: '*', schema: 'public', table: 'vote_speake
    .on('postgres_changes', { event: '*', schema: 'public', table: 'vote_agenda
    .on('postgres_changes', { event: '*', schema: 'public', table: 'vote_partic
    .on('postgres_changes', { event: '*', schema: 'public', table: 'vote_sessio
    .subscribe()

  return () => supabase.removeChannel(channel)
}, [load, session.id])

// Sync timer from DB
useEffect(() => {
  const channel = supabase.channel(`timer-${session.id}`)
    .on('broadcast', { event: 'timer_sync' }, ({ payload }) => {
      setTimerSeconds(payload.seconds)
      setTimerRunning(payload.running)
    }).subscribe()
  return () => supabase.removeChannel(channel)
}, [session.id])

useEffect(() => {
  if (timerRunning && timerSeconds > 0) {
    timerRef.current = setInterval(() => setTimerSeconds(s => s - 1), 1000)
  } else {
    clearInterval(timerRef.current)
  }
  return () => clearInterval(timerRef.current)

```

```

}, [timerRunning, timerSeconds])

// Summary countdown
useEffect(() => {
  if (showSummary) {
    summaryRef.current = setInterval(() => setSummaryCountdown(c => c - 1), 100)
  }
  return () => clearInterval(summaryRef.current)
}, [showSummary])

const activePoll = polls.find(p => p.status === 'active')
const calledSpeaker = speakers.find(s => s.status === 'called')
const waitingSpeakers = speakers.filter(s => s.status === 'waiting')
const activeAgenda = agenda.find(a => a.active)
const timerColor = timerSeconds > 60 ? 'timer-green' : timerSeconds > 30 ? 'tim
const closedPolls = polls.filter(p => p.status === 'closed')

const getVoteCount = (poll, type) => (poll.vote_responses || []).filter(v => v.
const getVotePercent = (poll, type) => { const t = (poll.vote_responses || []).

if (showSummary) return (
  <>
  <style>{css}</style>
  <div className="summary-screen">
    <div className="summary-title">Konference ukončena</div>
    <div className="summary-name">{session.title}</div>
    <div className="summary-grid">
      <div className="summary-stat"><div className="summary-stat-val" style={
        <div className="summary-stat"><div className="summary-stat-val" style={
          <div className="summary-stat"><div className="summary-stat-val" style={
        </div>
      </div>
      {closedPolls.length > 0 && (
        <div className="summary-resolutions">
          <div style={{ fontFamily:'JetBrains Mono,monospace', fontSize:9, colo
            {closedPolls.map((poll, i) => {
              const res = getResolutionResult(poll)
              const votes = poll.vote_responses || []
              return (
                <div key={poll.id} className="summary-resolution">
                  <div className="summary-resolution-num">#{i+1}</div>
                  <div className="summary-resolution-text">{poll.question}</div>
                  <div style={{ textAlign:'right' }}>
                    <div className={`summary-resolution-result ${res.cls}`}>{res.
                      <div style={{ fontFamily:'JetBrains Mono,monospace', fontSize
                        {votes.filter(v=>v.vote==='ano').length}:{votes.filter(v=>v
                      </div>
                    </div>
                  </div>
                </div>
              )
            })
          </div>
        </div>
      )
    </div>
  </div>
)

```

```

        </div>
    )
    }}}
</div>
))
<div className="summary-countdown">Obrazovka se uzavře za {summaryCountdo
</div>
</>
)

return (
    <>
        <style>{css}</style>
        <div className="display">
            <div className="display-header">
                <div className="display-logo">[<span>CTRL</span>]</div>
                <div style={{ fontFamily:'JetBrains Mono,monospace', fontSize:12, color
                <div className="participants-badge" style={{ marginLeft:'auto' }}>
                    <div className="participants-dot" />
                    <span className="participants-count">{participants.length} účastníků<
                </div>
            </div>
            <div className="display-content">
                <div className="display-main">
                    {activeAgenda && (
                        <div style={{ background:'rgba(42,107,255,0.05)', border:`1px solid
                        <div style={{ fontFamily:'JetBrains Mono,monospace', fontSize:10,
                        <div style={{ fontSize:22, fontWeight:800 }}>{activeAgenda.title}
                        {activeAgenda.description && <div style={{ fontSize:14, color:G.t
                        </div>
                    </div>
                </div>
            </div>
            {calledSpeaker && (
                <div className="display-called-card">
                    <div className="display-called-label"> Právě mluví</div>
                    <div className="display-called-name">{calledSpeaker.name}</div>
                    {calledSpeaker.note && <div style={{ fontSize:16, color:G.text2,
                    <div className={`timer-display ${timerColor}`}>{formatTimer(timer
                    </div>
                </div>
            </div>
            {activePoll && (
                <div className="result-card" style={{ border:`1px solid ${G.accent}
                <div style={{ fontFamily:'JetBrains Mono,monospace', fontSize:10,
                    <span style={{ animation:'pulse 1s infinite', display:'inline-b
                </div>
                <div className="result-question">{activePoll.question}</div>
                {[['ano',G.success,'ANO'], ['ne',G.danger,'NE'], ['zdrzuj_se',G.wa
                    <div key={type} className="result-bar-wrap">

```

```

        <div className="result-bar-label">
          <div style={{ fontSize:16, fontWeight:700, color }}>{label}>
            <div style={{ fontFamily:'JetBrains Mono,monospace', fontSi
          </div>
        <div className="result-bar-track">
          <div className="result-bar-fill" style={{ width:`${getVoteP
        </div>
        <div className="result-voters">
          {(activePoll.vote_responses||[]).filter(v=>v.vote===type).m
            <span key={v.id} className={`result-voter voter-${type.re
              )}}
            </div>
          </div>
        </div>
      )}}
    </div>
  )}
  {closedPolls.map((poll,i) => {
    const res = getResolutionResult(poll)
    return (
      <div key={poll.id} className="result-card">
        <div style={{ display:'flex', justifyContent:'space-between', a
          <div style={{ fontFamily:'JetBrains Mono,monospace', fontSize
            <div style={{ fontFamily:'JetBrains Mono,monospace', fontSi
          </div>
        <div style={{ fontSize:18, fontWeight:700, marginBottom:12 }}>{
          <div style={{ display:'flex', gap:20 }}>{
            <span style={{ fontFamily:'JetBrains Mono,monospace', fontSiz
              <span style={{ fontFamily:'JetBrains Mono,monospace', fontSiz
                <span style={{ fontFamily:'JetBrains Mono,monospace', fontSiz
              </div>
            </div>
          </div>
        </div>
      )
    )}}
  </div>
<div className="display-side">
  {reactions.length > 0 && (
    <div className="display-speaker-card">
      <div className="sec">REAKCE PUBLIKA</div>
      <div className="reactions-display">
        {reactions.map(r => (
          <div key={r.id} className="reaction-pill">
            <span style={{ fontSize:18 }}>{r.reaction==='souhlas'?':r.
              <span style={{ fontSize:12, color:G.text2 }}>{r.name}</span>
            </div>
          )}}
        </div>
      </div>
    </div>
  )}

```



```

    if (data) setSessions(data)
  }

const loadSession = useCallback(async (session) => {
  const [{ data: p }, { data: s }, { data: a }, { data: part }, { data: r }] =
    supabase.from('vote_polls').select('*', vote_responses('*')).eq('session_id',
    supabase.from('vote_speakers').select('*').eq('session_id', session.id).ord
    supabase.from('vote_agenda').select('*').eq('session_id', session.id).order
    supabase.from('vote_participants').select('*').eq('session_id', session.id)
    supabase.from('vote_participants').select('*').eq('session_id', session.id)
  ])
  if (p) setPolls(p)
  if (s) setSpeakers(s)
  if (a) setAgenda(a)
  if (part) setParticipants(part)
  if (r) setReactions(r)
}, [])

useEffect(() => { loadSessions() }, [])

useEffect(() => {
  if (!activeSession) return
  loadSession(activeSession)
  const ch = supabase.channel(`mod-${activeSession.id}`)
    .on('postgres_changes', { event: '*', schema: 'public', table: 'vote_polls'
    .on('postgres_changes', { event: '*', schema: 'public', table: 'vote_respon
    .on('postgres_changes', { event: '*', schema: 'public', table: 'vote_speake
    .on('postgres_changes', { event: '*', schema: 'public', table: 'vote_partic
    .subscribe()
  return () => supabase.removeChannel(ch)
}, [activeSession, loadSession])

// Timer countdown
useEffect(() => {
  if (timerRunning && timer > 0) {
    timerRef.current = setInterval(() => setTimer(t => { return t - 1 }), 1000)
  } else { clearInterval(timerRef.current) }
  return () => clearInterval(timerRef.current)
}, [timerRunning, timer])

// Broadcast timer to display
const broadcastTimer = useCallback(async (seconds, running) => {
  await supabase.channel(`timer-${activeSession?.id}`).send({
    type: 'broadcast', event: 'timer_sync', payload: { seconds, running }
  })
}, [activeSession])

```

```

const startTimer = () => { setTimerRunning(true); broadcastTimer(timer, true) }
const pauseTimer = () => { setTimerRunning(false); broadcastTimer(timer, false) }
const resetTimer = () => { setTimerRunning(false); setTimer(timerInput); broadcastTimer(timerInput, true) }
const setTimerAndReset = (s) => { setTimerInput(s); setTimer(s); setTimerRunning(true) }

const createSession = async () => {
  if (!newSession.title || !newSession.pin) return
  const { data } = await supabase.from('vote_sessions').insert([newSession]).select()
  if (data) { setSessions(prev => [data, ...prev]); setActiveSession(data); setTimerAndReset(newSession.pin) }
}

const createPoll = async () => {
  if (!newPoll.trim() || !activeSession) return
  await supabase.from('vote_polls').insert([
    { session_id: activeSession.id, question: newPoll.trim() }
  ]).select().then(({ data }) => {
    setNewPoll(''); loadSession(activeSession)
  })
}

const setPollStatus = async (pollId, status) => {
  if (status === 'active') await supabase.from('vote_polls').update({ status: 'active' }, { pollId })
  await supabase.from('vote_polls').update({ status, ...((status === 'closed' ? { pollId }) : {}) })
  loadSession(activeSession)
}

const callSpeaker = async (speaker) => {
  await supabase.from('vote_speakers').update({ status: 'called', called_at: new Date().toISOString() }, { speaker })
  setTimer(timerInput); setTimerRunning(false); broadcastTimer(timerInput, false)
  loadSession(activeSession)
}

const doneSpeaker = async (speakerId) => {
  await supabase.from('vote_speakers').update({ status: 'done' }, { speakerId })
  setTimerRunning(false); setTimer(timerInput); broadcastTimer(timerInput, false)
  loadSession(activeSession)
}

const addAgenda = async () => {
  if (!newAgenda.title || !activeSession) return
  const maxOrder = agenda.length > 0 ? Math.max(...agenda.map(a => a.order_num)) : 0
  await supabase.from('vote_agenda').insert([
    { session_id: activeSession.id, title: newAgenda.title, description: newAgenda.description, order_num: maxOrder + 1 }
  ]).select().then(({ data }) => {
    setNewAgenda({ title: '', description: '' }); loadSession(activeSession)
  })
}

const setActiveAgenda = async (agendaId) => {
  await supabase.from('vote_agenda').update({ active: false }, { agendaId })
  await supabase.from('vote_agenda').update({ active: true }, { agendaId })
  loadSession(activeSession)
}

```

```

const closeSession = async () => {
  if (!window.confirm('Opravdu ukončit konferenci? Tato akce je nevratná.')) re
  await supabase.from('vote_sessions').update({ active: false }).eq('id', activ
  // Generate PDF
  generatePDF(activeSession, polls, speakers, agenda, participants)
  setActiveSession(null); loadSessions()
}

const timerColor = timer > 60 ? G.success : timer > 30 ? G.warning : G.danger
const calledSpeaker = speakers.find(s => s.status === 'called')
const waitingSpeakers = speakers.filter(s => s.status === 'waiting')

if (!activeSession) return (
  <>
    <style>{css}</style>
    <div className="mod-wrap">
      <div className="mod-header">
        <div className="mod-logo">[<span>CTRL</span>]</div>
        <span className="mod-badge">MODERÁTOR</span>
      </div>
      <div style={{ padding:24, maxWidth:600 }}>
        <div className="sec">NOVÁ KONFERENCE</div>
        <input className="fi" placeholder="Název konference..." value={newSessi
        <input className="fi" placeholder="PIN pro účastníky (např. CTRL)" valu
        <button className="btn btn-p" onClick={createSession}>VYTVOŘIT KONFERENCE
        {sessions.length > 0 && (
          <div style={{ marginTop:28 }}>
            <div className="sec">EXISTUJÍCÍ KONFERENCE</div>
            {sessions.map(s => (
              <div key={s.id} style={{ display:'flex', alignItems:'center', gap
                <div style={{ flex:1 }}>
                  <div style={{ fontSize:14, fontWeight:700 }}>{s.title}</div>
                  <div style={{ fontFamily:'JetBrains Mono,monospace', fontSize
                </div>
                <span style={{ fontFamily:'JetBrains Mono,monospace', fontSize:
                  {s.active ? 'AKTIVNÍ' : 'UKONČENÁ'}
                </span>
                <span style={{ color:G.accent, fontSize:18 }}>></span>
              </div>
            ))}
            </div>
          </div>
        )
      </div>
    </div>
  )

```

```

return (
  <>
    <style>{css}</style>
    <div className="mod-wrap">
      <div className="mod-header">
        <div className="mod-logo">[<span>CTRL</span>]</div>
        <span className="mod-badge">MODERÁTOR</span>
        <div style={{ fontFamily:'JetBrains Mono,monospace', fontSize:12, color
        <div style={{ fontFamily:'JetBrains Mono,monospace', fontSize:11, color
        <div className="participants-badge" style={{ marginLeft:'auto' }}>
          <div className="participants-dot" />
          <span className="participants-count">{participants.length} účastníků<
        </div>
        <button className="btn btn-g" style={{ fontSize:10 }} onClick={() => se
        <button className="btn btn-d" style={{ fontSize:10 }} onClick={closeSes
        </div>

      <div className="mod-content">
        <div>
          {/* REACTIONS */}
          {reactions.length > 0 && (
            <div className="mod-panel">
              <div className="mod-title">REAKCE PUBLIKA ({reactions.length})</d
              {reactions.map(r => (
                <div key={r.id} className="reaction-mod-item">
                  <span style={{ fontSize:18 }}>{r.reaction==='souhlas'?':r.re
                  <span style={{ fontSize:13, fontWeight:600 }}>{r.name}</span>
                  <span style={{ fontSize:11, color:G.text2, marginLeft:'auto'
                </div>
              )))
            </div>
          )}

          {/* HLASOVÁNÍ */}
          <div className="mod-panel">
            <div className="mod-title">HLASOVÁNÍ</div>
            <input className="fi" placeholder="Otázka hlasování..." value={newP
            <button className="btn btn-p" onClick={createPoll}>+ VYTVOŘIT HLASO
            <div style={{ marginTop:16 }}>
              {polls.map((poll, i) => {
                const votes = poll.vote_responses || []
                const ano = votes.filter(v => v.vote==='ano').length
                const ne = votes.filter(v => v.vote==='ne').length
                const zdrzují = votes.filter(v => v.vote==='zdrzují_se').length
                const res = poll.status === 'closed' ? getResolutionResult(poll
                return (

```

```

<div key={poll.id} style={{ background:G.panel2, border:`1px
  <div style={{ fontFamily:'JetBrains Mono,monospace', fontSi
  <div style={{ fontSize:13, fontWeight:700, marginBottom:8 }
  <div style={{ display:'flex', gap:12, marginBottom:10, flex
    <span style={{ fontFamily:'JetBrains Mono,monospace', fon
    <span style={{ fontFamily:'JetBrains Mono,monospace', fon
    <span style={{ fontFamily:'JetBrains Mono,monospace', fon
    <span style={{ fontFamily:'JetBrains Mono,monospace', fon
  </div>
  {res && <div style={{ fontFamily:'JetBrains Mono,monospace'
  <div style={{ display:'flex', gap:8, flexWrap:'wrap' }}>
    {poll.status=== 'waiting' && <button className="btn btn-s"
    {poll.status=== 'active' && <button className="btn btn-d"
    {poll.status=== 'closed' && <span style={{ fontFamily:'Jet
  </div>
</div>
)
}})
</div>
</div>

{/* AGENDA */}
<div className="mod-panel">
  <div className="mod-title">PROGRAM JEDNÁNÍ</div>
  <input className="fi" placeholder="Název bodu..." value={newAgenda.
  <input className="fi" placeholder="Popis (volitelné)..." value={new
  <button className="btn btn-p" onClick={addAgenda}>+ PŘIDAT BOD</but
  <div style={{ marginTop:16 }}>
    {agenda.map(item => (
      <div key={item.id} className={`agenda-item${item.active?' activ
      <div className="agenda-num">{item.order_num}</div>
      <div style={{ flex:1 }}>
        <div style={{ fontSize:13, fontWeight:600 }}>{item.title}</
        {item.description && <div style={{ fontSize:11, color:G.tex
      </div>
      {item.active && <span style={{ fontFamily:'JetBrains Mono,mon
    </div>
    )}}
  </div>
</div>
</div>

<div>
  {/* TIMER */}
  <div className="mod-panel">
    <div className="mod-title">ČASOMÍRA</div>
    {calledSpeaker && (

```

```

    <div style={{ textAlign:'center', marginBottom:16, padding:16, ba
      <div style={{ fontFamily:'JetBrains Mono,monospace', fontSize:1
        <div style={{ fontSize:18, fontWeight:700, color:G.success }}>{
      </div>
    }}
    <div style={{ fontFamily:'JetBrains Mono,monospace', fontSize:64, f
    <div style={{ display:'flex', gap:8, justifyContent:'center', margi
      <button className="btn btn-s" onClick={startTimer} disabled={time
      <button className="btn btn-w" onClick={pauseTimer}>|| PAUZA</butt
      <button className="btn btn-g" onClick={resetTimer}>reset RESET</b
    </div>
    <div style={{ display:'flex', gap:8, alignItems:'center', flexWrap:
      <input className="fi" type="number" value={timerInput} onChange={
      <span style={{ fontFamily:'JetBrains Mono,monospace', fontSize:11
        [[60,120,180,300].map(s => (
          <button key={s} className="btn btn-g" style={{ fontSize:10, pad
            {s<60?`$${s}s`:`$${s/60}m`}
          </button>
        ))}
      </div>
    </div>
  </div>

  { /* SPEAKERS */ }
  <div className="mod-panel">
    <div className="mod-title">FRONTA ŘEČNÍKŮ ({waitingSpeakers.length}
    {waitingSpeakers.length===0 && <div style={{ color:G.text3, fontSiz
    {waitingSpeakers.map((s,i) => (
      <div key={s.id} style={{ display:'flex', alignItems:'center', gap
        <div style={{ fontFamily:'JetBrains Mono,monospace', fontSize:1
          <div style={{ flex:1 }}>
            <div style={{ fontSize:14, fontWeight:600 }}>{s.name}</div>
            {s.note && <div style={{ fontSize:11, color:G.text2, fontStyl
          </div>
            <button className="call-btn" onClick={() => callSpeaker(s)}>VYV
          </div>
        ))}
      {calledSpeaker && (
        <div style={{ padding:'14px 18px', background:'rgba(0,229,160,0.0
          <div style={{ fontFamily:'JetBrains Mono,monospace', fontSize:9
            <div style={{ fontSize:15, fontWeight:700, marginBottom:10 }}>{
              <button className="done-btn" onClick={() => doneSpeaker(calledS
            </div>
          </div>
        ))}
      {speakers.filter(s=>s.status==='done').length > 0 && (
        <div style={{ marginTop:16 }}>
          <div style={{ fontFamily:'JetBrains Mono,monospace', fontSize:9
            {speakers.filter(s=>s.status==='done').map(s => (

```

```

        <div key={s.id} style={{ padding:'8px 14px', color:G.text3, f
            )))
        </div>
    )}
</div>
</div>
</div>
</div>
</>
)
}

// -- MAIN --
export default function App() {
  const [view, setView] = useState('entry')
  const [sessionData, setSessionData] = useState(null)
  const [participantData, setParticipantData] = useState(null)

  if (view==='participant' && sessionData && participantData) return <Participant
  if (view==='display' && sessionData) return <DisplayScreen session={sessionData
  if (view==='moderator') return <ModeratorView />

  return (
    <>
      <style>{css}</style>
      <EntryScreen
        onParticipant={({ session, participant }) => { setSessionData(session); s
        onModerator={() => setView('moderator')}
        onDisplay={(session) => { setSessionData(session); setView('display') }}
      />
    </>
  )
}

```